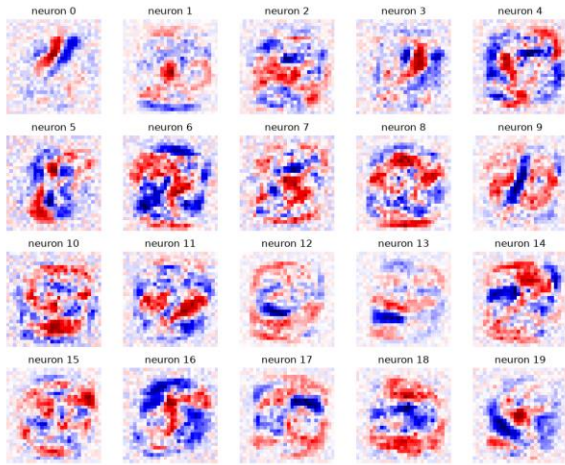


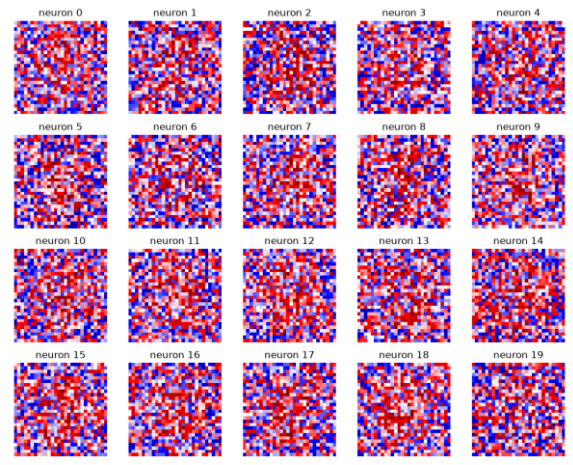
NK model vs MNIST for autoencoders on an MLP

The MLP structure is 784 inputs, 1 hidden layer of 20 neurons, and an output layer of 10 neurons. For the autoencoder, each hidden layer neuron also has its own 784 decoder neurons, each with weights and biases. Task loss was computed as Crossentropy loss, and reconstruction calculated as MSE loss. Task loss was backpropagated to only the output layer weights and biases (does not affect hidden layer weights and biases), and reconstruction loss was backpropagated through decoder weights and biases, and hidden layer weights and biases. The optimiser was SGD, with a learning rate of 0.1. The training cycle was that for each batch, the cycle was either reconstruction or task with 50/50 probability. For reconstruction, MSE loss was calculated from decoder neurons and backpropagated. For task, Crossentropy is calculated and backpropagated. The batch size was 64.

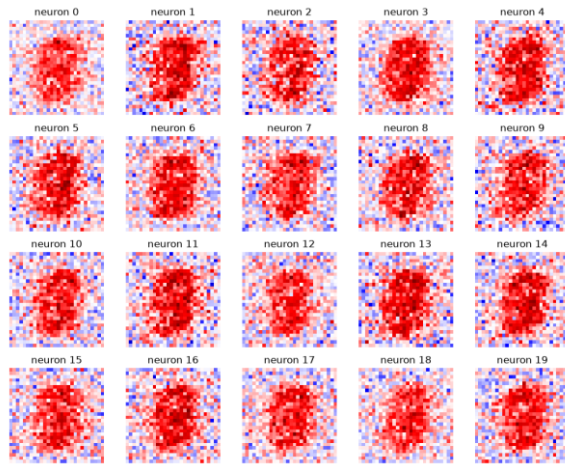
The MLP model without the autoencoder on average achieves ~95% test set accuracy. With an autoencoder as described above, it achieves ~40 - 45% test set accuracy after 10 epochs and around 50 – 55% after 20 epochs. Figure 1 shows the hidden layer weights for standard MLP and MLP with neuron autoencoder (NaN), and the decoder weights and biases for the MLP with autoencoder, all after 10 epochs of training.



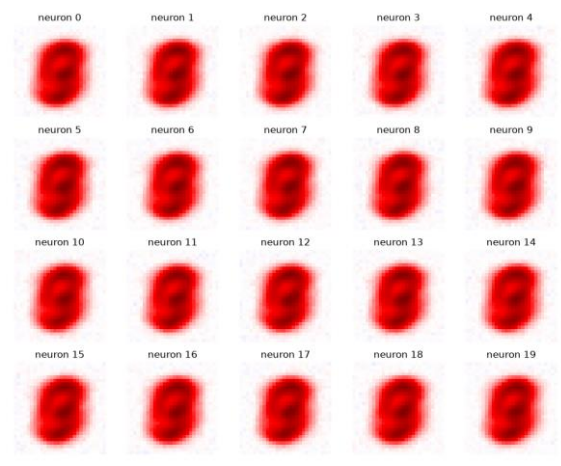
(a)



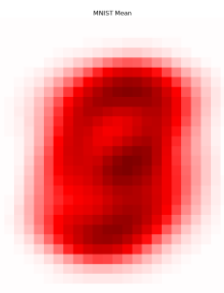
(b)



(c)



(d)



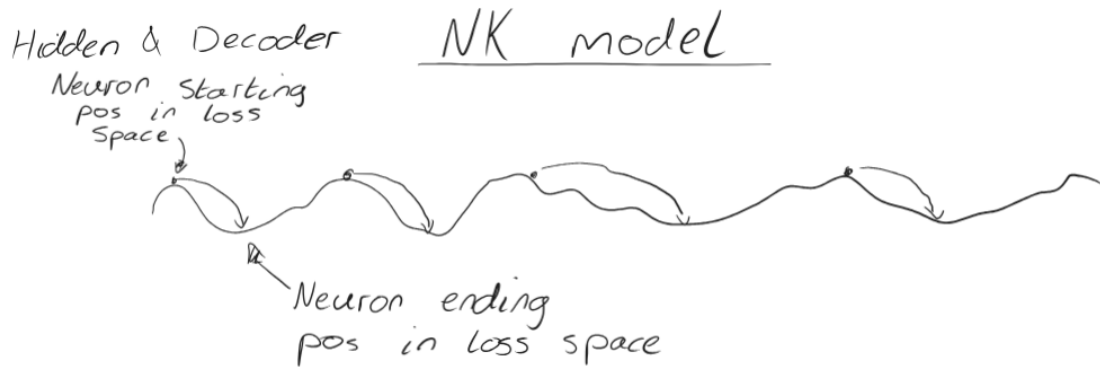
(e)

Figure 1: normalised graphs for weights and biases where red is positive and blue is negative (a): incoming weights to the 20 hidden layer neurons for default MLP. (b): same weights for MLP with NaN. (c): outgoing weights from 20 hidden layer neurons to their individual 28x28 decoder neurons. (d): biases for the 28x28 decoder neurons for each of the 20 hidden layer neurons. (e): the actual MNIST mean

Note with figure 1 all the images are normalised. Although Figure 1(b) looks like lots of random large values, on average the largest value is 0.048, while for Figure 1(a), the average largest weight value is 0.414.

The best I can reason as to why the NK model worked so well, but MNIST doesn't for this architecture, is that NK produces an arbitrary function across an N dimensional plane that, on *average*, has the same "height" everywhere, the inputs are random, independent bits. This produces a recon loss landscape with lots of local minima at similar losses. With the stochastic starting points, each neurons weights and biases find *unique* minima of similar quality that maximally represents the input data in a *different* non-linear way such that the output layer can use to effectively reproduce the function defined by the NK model. The important point is the hidden layer produces a diverse, input preserving basis that the output layer can use to fit the NK function as a linear combination of those non-linear features.

With MNIST, the function space is not evenly distributed, the average is not the same across the 28x28 grid. On average, MNIST is a white blob in the middle surrounded by black. The decoder weights and biases all see a steep valley towards the same minima, which is a blob in the middle with nothing around it. Particularly the biases, the easiest path for the network to take is to have the hidden layer send essentially a blank canvas, and the biases of the decoders add on the average for the MNIST dataset. With the decoder biases doing most of the work, the hidden layer weights don't learn to maximise information, and they retain their small, random starting weights. Figure 2 shows professional drawings of how I imagine the loss space for both datasets.



MNIST

Decoder neuron
loss space

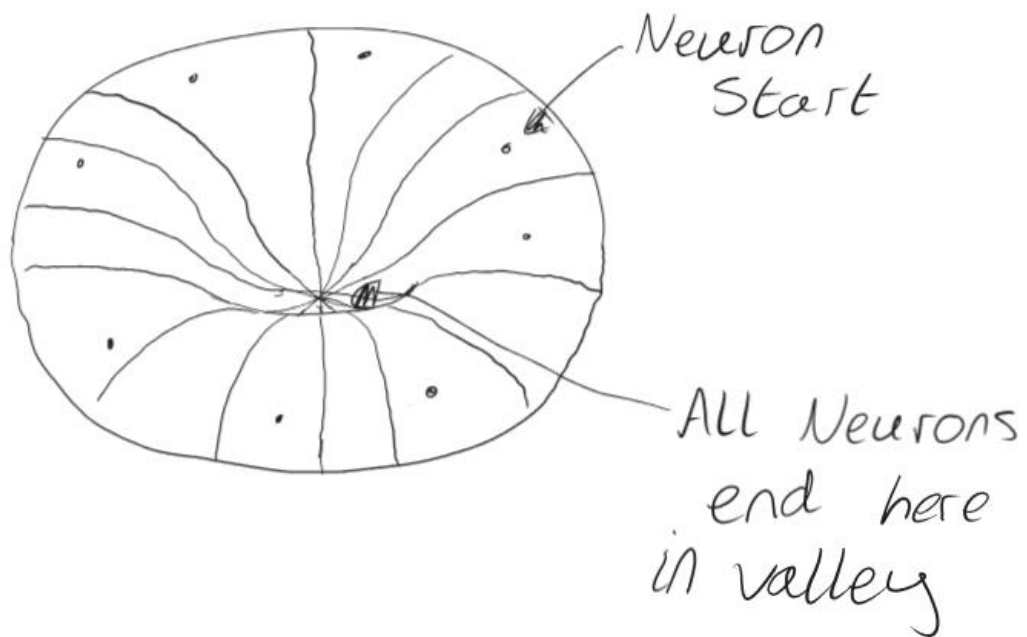


Figure 2: what I imagine the loss landscape looking like

For ways to resolve the issue, I tried subtracting the mean from the images which removes the large valley, however, accuracy seems to remain approximately the same of around 50-55% after 20 epochs. Since training on the autoencoder or on the task is done as a batch, batch size impacts training performance, decreasing batchsize from 64 down to 8 increased test set accuracy to approximately 65% after 20 epochs though more work needs doing on this. Next steps are trying to train the autoencoder layer first

till that stabilises, then freezing that and training the output layer (with the mean subtracted as otherwise, they fall into the valley as before). An idea I will try as well is to see what happens by adding some diversity pressure, I will look into how this could be implemented in a biologically plausible way, though I will play around with several forms. It would also be interesting to see how this effects convolutional layers.

Code for the MLP with NaN can be found on my github:

https://github.com/Isaac-J-Gardner/SoE-Summer-Internship-Neurons-that-Learn-on-their-Own/blob/main/MLP_MNIST/MLP_NaN_v1/MNIST_MLP_NaN.py